National Institute of Technology Rourkela

Department of Computer Science and Engineering

Master of Technology in Computer Science

LABORATORY ASSIGNMENT REPORT

Advanced Data Structure Laboratory (CS 6173)

Assignment-I

Simulation of coin tossing, performance analysis of bubble, quick and insertion sorting

SUBMITTED BY:
Name: Sumit Karmakar
Roll No: 226CS1008
Branch: Computer Science & Engineering
Semester: 1st

SUBMITTED TO:
Dr. Bibhudatta Sahoo
Department of Computer Science & Engineering
Date of Submission: 29th August, 2026

1. EXPERIMENT OVERVIEW

For this assignment we have performed simulation of coin tossing. We have used random number generator for simulating the coin tossing event.

We have simulated comparison-based performance analysis of standard bubble sort and bubble sort with early stopping mechanism for different input classes (Sorted, nearly sorted, highly inversional).

We compared insertion sort and quick sort, then we design a hybrid sort where for subproblem size of 12 or less we do insertion sort and for bigger problem we do standard partition-based quicksort. A comparison-based study has also been performed to understand how merging two different type of sorting we can achieve better execution time for sorting of a given dataset.

2. OBJECTIVES

- To simulate coin toss with random generator in C++.
- To get performance-based analysis of bubble sort and bubble sort with early stopping.
- To compare insertion sort and quick sort and design a hybrid sort that utilizes benefits of both the sorting methods.
- Plotting of output data in MATLAB to get visual representation and performance analysis.

3. THEORY & BACKGROUND

For coin tossing we make use of inbuilt random number generator and uniform distribution provided by C++.

Bubble sort is a comparison-based sorting algorithm which runs in $O(n^2)$ time complexity. Classical Bubble Sort always runs n – 1 time where n is the size of the given input. We can include an early stopping mechanism, which checks whether we need to go to next pass or not by checking that data is already sorted or not using a swap checking variable.

4. IMPLEMENTATION / CODE

Below is the C++ implementation of the assigned questions:

- Q1: Coin Tossing

```cpp
#include<iostream>
#include<random>
#include<fstream>

void oneCoinToss(std::mt19937 generator){
    std::uniform_int_distribution<int> coin(0,1);
    int N = 5000;

    std::fstream file("oneCoinToss.csv", std::ios::out| std::ios::in | std::ios::trunc);
    if(!file.is_open()){
        throw std::runtime_error("Failed to open the file!!");
    }
    file << "Toss,Heads,Tails,HeadProbability,TailProbability";
    file << "\n";

    int heads = 0, tails = 0;

    for(int i = 1; i <= N; i++){
        int result = coin(generator);
```

```
            if(result){
                heads++;
            }
            else tails++;

            double head_prob = heads*1.0/i;
            double tail_prob = tails*1.0/i;

            file << i << ", " << heads << ", "
            << tails << ", " << head_prob << ", " << tail_prob << "\n";
        }
        file.close();
        std::cout<<heads << " " << tails << " " <<heads*1.0/N << " " << tails*1.0/N << std::endl;

    }

    void twoCoinToss(std::mt19937 generator){
        std::uniform_int_distribution<int> coin(0,1);

        int N = 5000;

        std::fstream file("twoCoinToss.csv", std::ios::out| std::ios::in | std::ios::trunc);
        if(!file.is_open()){
            throw std::runtime_error("Failed to open the file!!");
        }
        file << "Toss,HH,HT,TH,TT,P(HH),P(HT),P(TH),P(TT)";
        file << "\n";

        int HH = 0, HT = 0, TH = 0, TT = 0;

        for(int i = 1; i <= N; i++){
            int result1 = coin(generator);
            int result2 = coin(generator);
            if(result1){
               if(result2){
                    HH++;
               }
               else HT++;
            }
            else{
                if(result2){
                    TH++;
                }
                else TT++;
            }

            double HH_prob = HH*1.0/i;
            double HT_prob = HT*1.0/i;
```

```
            double TH_prob = TH*1.0/i;
            double TT_prob = TT*1.0/i;

            file    << i << ", " << HH << ", " << HT << ", " << TH << ", " << TT << ", "
                    << HH_prob << ", " << HT_prob << ", " << TH_prob << ", " << TT_prob << "\n";
        }
        file.close();

        std::cout<<HH << " " << HT << " " << TH << " " << TT << " " << HH*1.0/N << " " << HT*1.0/N << "
    " <<TH*1.0/N << " "<<TT*1.0/N << "\n";

    }
    int main(){
        std::random_device rd;
        std::mt19937 generator(
        static_cast<unsigned int>(rd())
        );

        // Experiment with single coin toss;
        oneCoinToss(generator);

        // Experiment with 2 coin tosses;
        twoCoinToss(generator);

        return 0;

    }
```

- Q2: Performance analysis of Bubble Sort

```
#include "generation/RandomInputGenerator.hpp"
#include "sorting/Sorting.hpp"
#include <iostream>
#include <fstream>
#include <algorithm>

int bSortWithoutEarlyStopping(std::vector<int>& data){
    size_t size = data.size();
    int comp = 0;
    for(int i = 1; i < size; i++){
        for(int j = 0; j < size - i; j++){
            if(data[j] > data[j+1]){
                std::swap(data[j], data[j+1]);
            }
            comp++;
        }
```

```
    }
    return comp;
}
int main(){
    RandomInputGenerator generator;
    BubbleSort bSortWithtEarlyStopping;
    // highly randomly ordered data;
        std::fstream file1("comparisonHighlyRandom.csv", std::ios::in | std::ios::out | std::ios::trunc);
    if(!file1.is_open()) throw std::runtime_error("File not openeed!!");

    file1<< "InputSize, Comp(NES), Comp(ES)" << "\n";

    int cmpWithoutEarlyStopping = 0, cmpWithEarlyStopping = 0;
    for(int datasize = 1; datasize <= 100; datasize++){
                std::vector<int> b1Data = generator.generateInput(datasize, 1, 1000, InputType::HIGHLY_INVERSIONAL);
        std::vector<int> b2Data= b1Data;
        cmpWithEarlyStopping = bSortWithtEarlyStopping.sort(b1Data);
        cmpWithoutEarlyStopping = bSortWithoutEarlyStopping(b2Data);
         file1 << datasize << ", " << cmpWithoutEarlyStopping << ", " << cmpWithEarlyStopping << "\n";
    }

    file1.close();

    // nearly sorted data;
        std::fstream file2("comparisonNearlySorted.csv", std::ios::in | std::ios::out | std::ios::trunc);
    if(!file2.is_open()) throw std::runtime_error("File not openeed!!");

    file2<< "InputSize, Comp(NES), Comp(ES)" << "\n";

    cmpWithoutEarlyStopping = 0, cmpWithEarlyStopping = 0;
    for(int datasize = 1; datasize <= 100; datasize++){
                std::vector<int> b1Data = generator.generateInput(datasize, 1, 1000, InputType::NEARLY_SORTED, SortOrder::ASCENDING,0.08);
        std::vector<int> b2Data= b1Data;
        cmpWithEarlyStopping = bSortWithtEarlyStopping.sort(b1Data);
        cmpWithoutEarlyStopping = bSortWithoutEarlyStopping(b2Data);
         file2 << datasize << ", " << cmpWithoutEarlyStopping << ", " << cmpWithEarlyStopping << "\n";
    }

    file2.close();

    // sorted data;
    std::fstream file3("comparisonSorted.csv", std::ios::in | std::ios::out | std::ios::trunc);
```

```
    if(!file3.is_open()) throw std::runtime_error("File not openeed!!");

    file3<< "InputSize, Comp(NES), Comp(ES)" << "\n";

    cmpWithoutEarlyStopping = 0, cmpWithEarlyStopping = 0;
    for(int datasize = 1; datasize <= 100; datasize++){
        std::vector<int> b1Data = generator.generateInput(datasize, 1, 1000, InputType::SORTED);
        std::vector<int> b2Data= b1Data;
        cmpWithEarlyStopping = bSortWithtEarlyStopping.sort(b1Data);
        cmpWithoutEarlyStopping = bSortWithoutEarlyStopping(b2Data);
         file3 << datasize << ", " << cmpWithoutEarlyStopping << ", " << cmpWithEarlyStopping <<
"\n";
    }

    file3.close();

    return 0;
}
```

- Q3: Variants of QUICK SORT

```
#include "generation/RandomInputGenerator.hpp"
#include "sorting/Sorting.hpp"
#include <vector>
#include <algorithm>
#include <fstream>

class HybridSort{
private:
    static constexpr int CUTOFF = 12;
    static long long comparisons;

    static int lomutoPartition(
    std::vector<int>& data,
    int start,
    int end
    ) {

        int pivot = data[end];

        int left = start - 1;

        for (int right = start; right < end; right++) {


            comparisons++;
```

```
            if (data[right] < pivot) {

                left++;

                std::swap(data[left], data[right]);
            }
        }

        std::swap(data[left + 1], data[end]);

        return left + 1;
    }

    static void hybridSort(
        std:: vector<int>& data,
        int start,
        int end
    ){
        if (start >= end)
            return;

        int size = end - start + 1;

        // small problem -> insertion sort
        if(size <= CUTOFF){
            comparisons += InsertionSort::sort(data,start,end);
            return;
        }

        int p = lomutoPartition(data,start,end);

        hybridSort(data, start, p-1);
        hybridSort(data, p+1,end);

    }

    public:
        static void sort(std::vector<int>& data){
            if(data.empty())
                return;

            hybridSort(data,0,data.size()-1);
        }

        static void resetComparisons()
        {
            comparisons = 0;
```

```
        }


        static long long getComparisons()
        {
            return comparisons;
        }

};

long long HybridSort::comparisons = 0;

int main(){

    RandomInputGenerator generetor;

    std::fstream file1("iqSortComparison.csv", std::ios::in | std::ios::out | std::ios::trunc);
    if(!file1.is_open()) throw std::runtime_error("File not opened!!");

    file1<< "InputSize, Comp(ISort), Comp(QSort)" << "\n";
    for(int datasize = 10; datasize <= 1000; datasize++){
                        std::vector<int>     iSortData     =     generetor.generateInput(datasize,     1,
9999,InputType::HIGHLY_INVERSIONAL,SortOrder::ASCENDING, 0.75);
        std::vector<int> qSortData = iSortData;

        int iSortComparison = InsertionSort::sort(iSortData);

        QuickSort::sort(qSortData, 0, qSortData.size()-1, PartitionScheme::HOARE);
        int qSortComparison = QuickSort::getComparisons();
        QuickSort::resetComparisons();

        file1<< datasize << ", " << iSortComparison << ", " << qSortComparison << "\n";
    }
    file1.close();

    std::fstream file2("qhSortComparison.csv", std::ios::in | std::ios::out | std::ios::trunc);
    if(!file2.is_open()) throw std::runtime_error("File not opened!!");

    file2<< "InputSize, Comp(QSort), Comp(HQSort)" << "\n";
    for(int datasize = 100; datasize <= 400; datasize++){
                        std::vector<int>     qSortData     =     generetor.generateInput(datasize,     1,
9999,InputType::HIGHLY_INVERSIONAL,SortOrder::ASCENDING, 0.75);
        std::vector<int> hybridData = qSortData;

        QuickSort::sort(qSortData, 0, qSortData.size()-1, PartitionScheme::LOMUTO);
        int qSortComparison = QuickSort::getComparisons();
        QuickSort::resetComparisons();
```

```
        HybridSort::sort(hybridData);
        int hSortComparison = HybridSort::getComparisons();
        HybridSort::resetComparisons();
        file2<< datasize << ", " << qSortComparison << ", " << hSortComparison << "\n";
    }
    file2.close();

    return 0;
}
```

5. SAMPLE OUTPUT & SIMULATION RESULTS

- Q1: Coin Toss (Single and double-coin tossing simulation):
  We have simulated for 5000 times for both single and double coin tossing.

Experiment	P(Head)	P(Tail)	Remarks
Single coin toss	0.5154	0.4846	Both converge near to 0.5 line

Experiment	P(H H)	P(H T)	P(T H)	P(T T)	Remarks
Double coin toss	0.2520	0.2532	0.2554	0.2394	All 4 lines converge near to 0.25 line

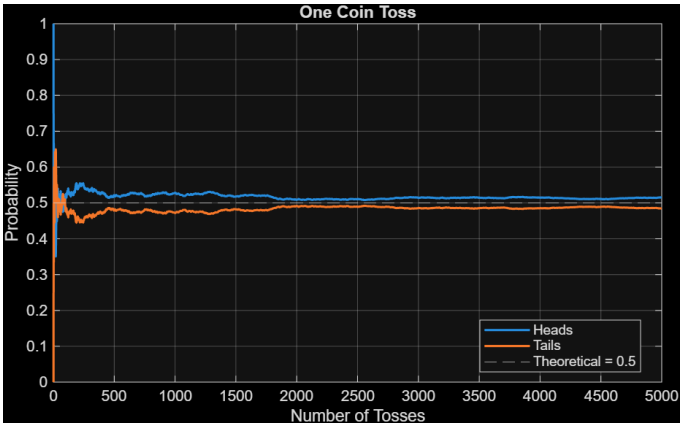

Fig 1: One coin toss simulation

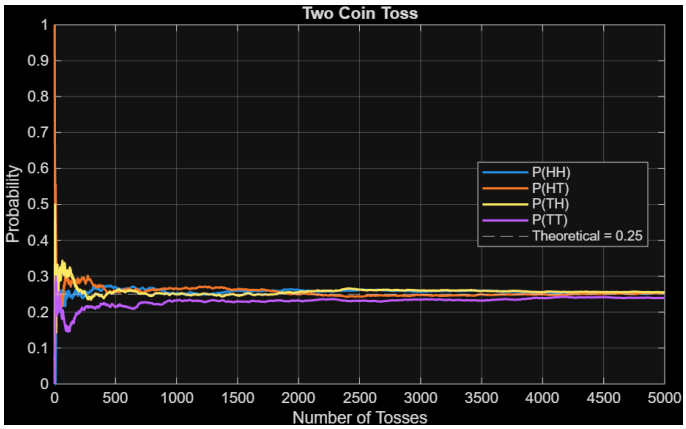

Fig 2: Two coin toss simulation

- Q2: Performance Analysis of Two Variants of Bubble Sort:
  We take different input classes (sorted, nearly sorted and highly inversional) with different input size and get the number of comparisons that are performed by two variants of bubble sort.

Input size	Input Class	Classical Bubble Sort	Bubble Sort with early stopping	Remarks
10	Sorted	36	9	Bubble sort with early Stopping performs well for datasets with sorted or nearly sorted subparts than the classical bubble sort
	Nearly Sorted	36	15	
	Highly Inversional	36	35	
25	Sorted	276	25	
	Nearly Sorted	276	255	
	Highly Inversional	276	261	
50	Sorted	1176	49	
	Nearly Sorted	1176	1040	
	Highly Inversional	1176	1175	
100	Sorted	4950	99	
	Nearly Sorted	4950	4515	
	Highly Inversional	4950	4935	

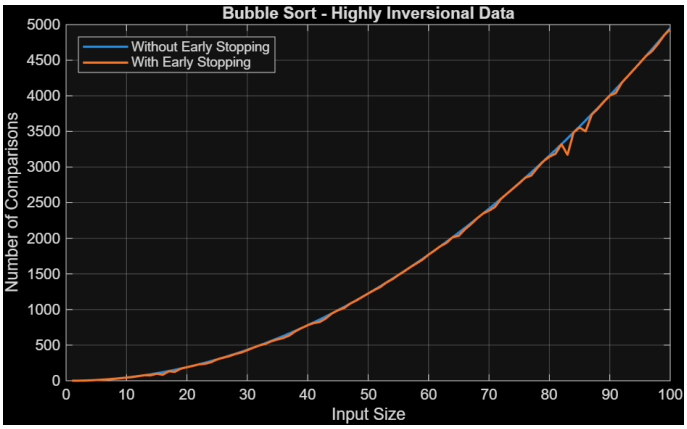

Fig 3: Comparison for highly random data

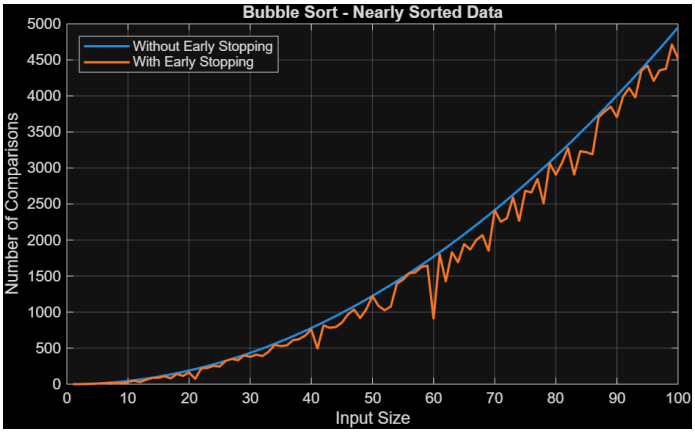

Fig 4: Comparison for nearly sorted data

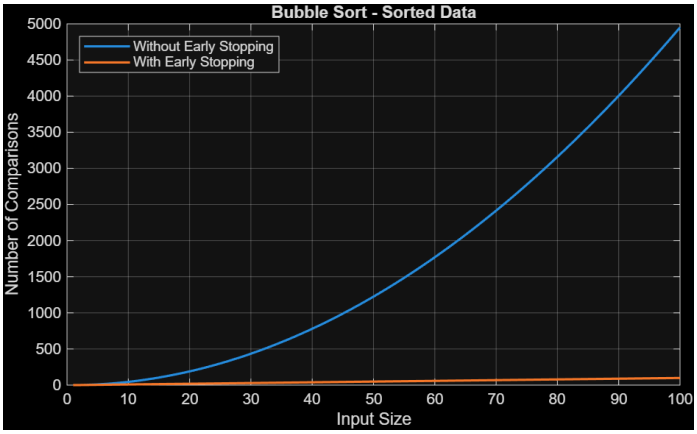

Fig 5: Comparison for sorted data

- Q3: Variants of Quick Sort:
  Comparison based performance analysis for insertion and quick sort for different size inputs.

Input size	Input Class	Insertion Sort	Quick Sort	Remarks
10	Highly Inversional	38	48	Insertion sort performs Better for lesser sized input. But as input size increases quick sort performs relatively well than the insertion sort.
100	Highly Inversional	3561	1039	
500	Highly Inversional	75534	6373	
1000	Highly Inversional	276750	14044	

Time complexity (apriori : order of magnitude method) table for insertion and quick sort.

Algorithm	Best Case	Average Case	Worst Case
Quick Sort	O(N log N)	O(N log N)	$O(N^2)$
Insertion Sort	O(N)	$O(N^2)$	$O(N^2)$

Comparison based performance analysis for hybrid and quick sort for different size inputs.

Input size	Input Class	Quick Sort	Hybrid (Quick + Insertion)	Remarks
100	Highly Inversional	697	515	Hybrid sort performs Better than quick sort As it incorporates insertion sort for lower size (<=20) subproblems.
200	Highly Inversional	1556	1213	
300	Highly Inversional	2780	2201	
400	Highly Inversional	3687	3005	

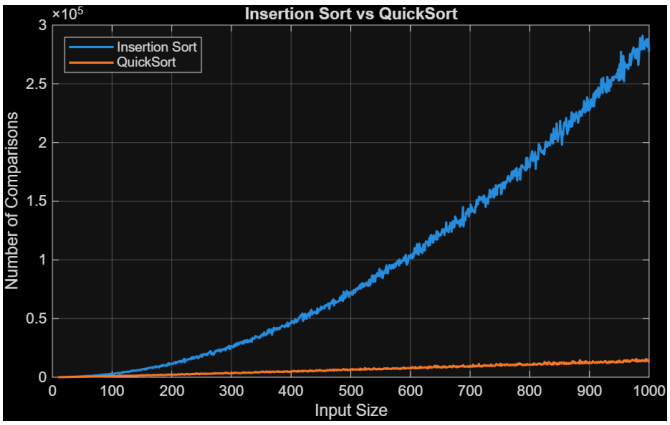

Fig 6: Insertion Sort and Quick Sort comparison

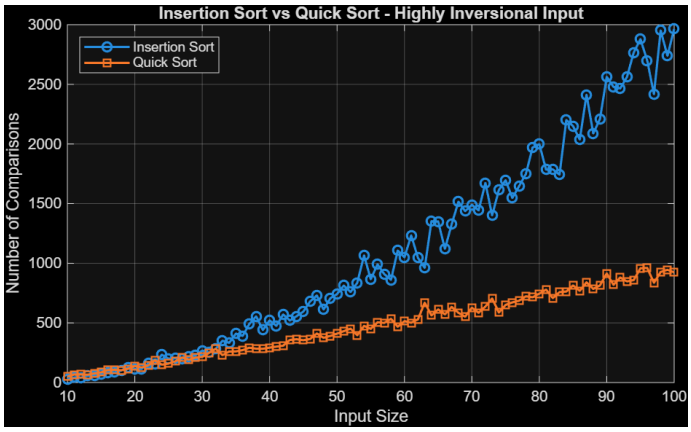

Fig 7: Insertion Sort and Quick Sort comparison for smaller input size.

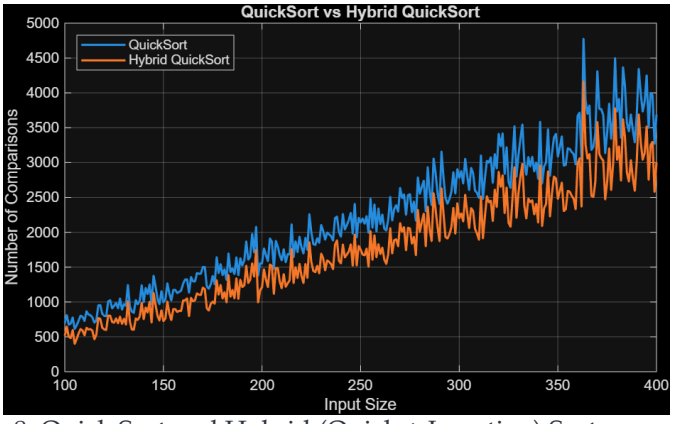

Fig 8: Quick Sort and Hybrid (Quick + Insertion) Sort comparison

6. OBSERVATIONS & ANALYSIS

Q1: Coin Tossing:

Using C++ random number generator and uniform distribution (0,1), to simulation coin tossing. Probability of getting head is 0.5 theoretically, and for two-coin toss every combination (HH, HT, TH, TT) has a probability of 0.25. Simulation the coin toss using random number generator we get empirical values of these probabilities of outcomes and that are very close to actual theoretical probability. Initial trends show huge variance from the 0.5 line (one coin toss) and 0.25 line (two-coin toss), but as the number of tosses increases it slows converges towards the theoretical expected value.

Q2: Bubble sort Performance analysis:

Classical Bubble Sort performs $n(n-1)/2$comparisons in the implementation used in this experiment, giving a time complexity of $O(n^2)$. The early-stopping variant can terminate before completing all passes when no swaps occur during a pass. In this question, we compared both versions of Bubble Sort for different input classes. By analysing the output file and visualizing in MATLAB we can confirm that Bubble sort with this early stopping mechanism performs well in cases where the number of inversion pair is less in the given data.

Q3: Quick sort performance analysis:

Quick sort is Divide and Conquer based approach, which uses partition-based technique to sort an input data. Classical quick sort goes on dividing the data till there is a single element left before the conquer part. Quick sort is an in-place, unstable, Quick Sort is an in-place, comparison-based sorting algorithm based on the divide-and-conquer strategy. Its average time complexity is $O(n \log n)$, while its worst-case complexity is $O(n^2)$, depending on the partitioning behaviour.

The comparison-based analysis shows that Insertion Sort performs competitively for small input sizes, whereas Quick Sort becomes more efficient as the input size increases. This is mainly because Quick Sort reduces the problem into smaller subproblems through partitioning, while Insertion Sort requires more comparisons as the input size grows.

Based on this observation, a hybrid sorting algorithm was implemented. For subproblems with size 12 or less, Insertion Sort is used; for larger subproblems, the standard partition-based Quick Sort is applied. The experimental results show that the hybrid approach performs fewer comparisons than the standard Quick Sort for the tested highly inversional inputs.

7. CONCLUSION

In this assignment, I studied random input generation, comparison-based analysis of sorting algorithms, and hybrid sorting techniques. The experiments helped me understand how different input characteristics affect the performance of sorting algorithms.

The coin-toss simulation showed how empirical probabilities converge towards their theoretical values as the number of trials increases. The Bubble Sort experiment demonstrated the advantage of an early-stopping mechanism for sorted and nearly sorted inputs. The comparison between Insertion Sort and Quick Sort showed that different algorithms can perform better for different input sizes.

Finally, the hybrid sorting experiment showed how combining Insertion Sort with Quick Sort can reduce the number of comparisons for the tested inputs. MATLAB was used to visualize the experimental results and make the comparison easier to analyse.

I've also structured the C++ implementation using separate modules and a Git repository so that the components developed in this laboratory can be reused in future experiments.

8. STANDARDS:

C++ 23, g++ 16.2.0

9. CODE REPO LINK

Github repo link (click on the icon)

10. SOFTWARE USED:

vscode , MinGW, cMake, MATLAB, Grammarly .

11. REFERENCE:

1. **cppreference.com**, *Random Number Generation Library*, documentation for C++ <random>, std::uniform_int_distribution, and random number generation facilities.
   C++ <random> reference
2. **MathWorks**, *readtable — Create table from file*, MATLAB documentation. Used for importing CSV files for analysis.
3. **MathWorks**, *plot — 2-D line plot*, MATLAB documentation. Used for visualizing the experimental results.
4. **MathWorks**, *Plots That Support Tables*, MATLAB documentation. Used as a reference for plotting data imported into MATLAB tables.
5. **Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C.**, *Introduction to Algorithms*, MIT Press. Reference for sorting algorithms, divide-and-conquer techniques, and asymptotic analysis.

12. CREDIT:

- **C++ Documentation:** cppreference.com was referred to for understanding C++ random number generation and standard library functionality.
- **MATLAB Documentation:** MathWorks documentation was referred to for CSV data import and plotting.
- **Code Development:** The implementation, experimentation, data generation, comparison counting, CSV generation, and analysis were performed by me.
- **Writing Assistance:** Grammarly was used for basic grammar and language correction.
- **Development Environment:** Visual Studio Code, MinGW/G++, CMake and MATLAB were used during the implementation and analysis.